CS50 · SELF-STUDY REVIEW BLUEPRINT

Data Structures & Algorithms One-Page Review Sheet

Everything you've practiced, compressed into a reference card you can revisit anytime: one core template per topic, a bug comparison table, and a few easy-to-medium exercises. Explains the "why" like a tutor, not just the answers.

`Python 3` `Tutor style` `Includes Java / AP CSA analogies` `Includes edge cases`

---

00 Review Order

From "how data is organized" to "how data gets processed." Each box is the foundation for the next one — don't skip ahead.

01 **dict/list/set patterns** Group • Count • Sort	02 **Stack / Queue** LIFO / FIFO	03 **Linked List** Pointer traversal	04 **Recursion** Base case / shrink problem	05 **Binary Search** Sorted • halving
06 ← you are here **Sorting** Selection/Insertion/Merge	07 **Tree** Recursion extended	08 **Hash Table problems** dict, leveled up	09 **File / CSV / JSON** Real-world data	10 **Graph** BFS / DFS

A pacing note for you: it's completely normal that merge sort doesn't feel solid yet — it stacks "recursion + merging" on top of each other. First make sure you can independently write binary search and factorial (the smallest versions of recursion + halving), then come back to merge sort — it'll click much faster. Tree comes right after sorting because it's essentially "recursion + node pointers" fused together — you've already practiced half of each piece.

01 dict / list / set Patterns

90% of data-analysis problems are a combination of these three moves: **group** the data, **count/accumulate**, then **sort**. Recognizing the pattern matters more than memorizing code.

Core Templates

Three big patterns — count / accumulate / group — pattern

```python
# 1) Count: how many times each key appears
counts = {}
for item in records:
    counts[item] = counts.get(item, 0) + 1   # .get avoids KeyError

# 2) Accumulate: sum up values for the same key
totals = {}
for user, days in records:
    totals[user] = totals.get(user, 0) + days

# 3) Group: collect records sharing a key into one list
groups = {}
for name, subject, score in records:
    groups.setdefault(name, []).append(score)   # auto-creates an empty list the first time
```

Sort + find the top + handle ties — sorted

```python
# Sort by value, highest first; item is a (key, value) tuple
ranked = sorted(averages.items(), key=lambda item: item[1], reverse=True)

# Find the single highest scorer
top = max(averages, key=averages.get)

# Handle ties for first place: get the max value, then collect all keys matching it
hi = max(averages.values())
winners = [k for k, v in averages.items() if v == hi]
```

Tutor tip: don't memorize `key=lambda item: item[1]` — understand it. `.items()` gives you a stream of `(key, value)` tuples, and `item[1]` is the value. In plain words: "tell sorted to compare using the 2nd element of each tuple." Swap `item[1]` for `item[0]` and it sorts by key instead — try it once and it sticks.

Java / AP CSA analogy — Python dict ≈ HashMap<K,V>; `counts.get(k, 0)+1` ≈ `map.getOrDefault(k,0)+1`; set ≈ HashSet; list ≈ ArrayList.

Bug Comparison Table

WRONG	WHY IT'S WRONG	CORRECT
`d[k] += 1`	First time key k appears, d[k] doesn't exist → KeyError	`d[k] = d.get(k,0)+1`
`for k in d:` then `d[k]`	Works, but wordy and easy to lose track of the value	`for k,v in d.items():`
`sorted(d)` hoping to sort by score	Defaults to sorting by key (name), not score	`sorted(d.items(), key=lambda x:x[1])`
Just take `max()` as "the winner"	Misses people when there's a tie	Find the max value first, then filter everyone matching it

Edge Cases

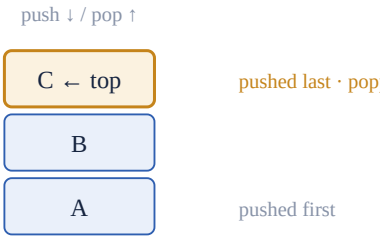
`Empty records → return {} or 0, don't error` `Division by zero (count is 0 when averaging)`
`Ties for first place` `Inconsistent key casing`

◆ EXERCISES

1. `Easy` Given a list of `(name, subject, score)`, return how many subjects each student has (use the count template).
2. `Easy` Count how often each subject appears, and sort output from most to least frequent.
3. `Medium` Compute each student's average score, and return the student with the highest average (handle ties correctly).
4. `Medium` Find all failing records (score < 70), then sort by score ascending.
5. `Medium` Use a set to collect every subject that appears, and find "the highest score in each subject."

02 Stack

LIFO — last in, first out. Think of a stack of plates: you can only take from the top. Python simulates it directly with a list.



`append()` to push · `pop()` to pop · `stack[-1]` to peek the top

Core Template: Bracket Matching

is_balanced — the classic stack application — stack

```python
def is_balanced(text):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in text:
        if ch in "([{":
            stack.append(ch)              # push opening bracket
        elif ch in ")]}":
            if not stack:                 # ① check empty FIRST, or pop() errors
                return False
            if stack.pop() != pairs[ch]:  # ② popped value must match this closer
                return False
        # any other character is simply skipped
    return not stack                      # ③ stack must be empty at the end
```

Three-step checklist: ① Closing bracket arrives but stack is empty → an extra closer → False. ② Popped opener doesn't match → wrong type → False. ③ Scan finishes but stack still has stuff left → an unclosed opener → False. Miss any one of these three checks and you've got a bug.

Level Up: Locating the Error (store index)

`check_brackets — stack stores (char, index)` `tuple`

```python
def check_brackets(text):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for index, ch in enumerate(text):        # enumerate gives you position too
        if ch in "([{":
            stack.append((ch, index))         # key move: store the position alongside
        elif ch in ")]}":
            if not stack:
                return f"Extra closing bracket at index {index}"
            top_char, top_index = stack.pop() # tuple unpacking
            if top_char != pairs[ch]:
                return f"Mismatched bracket at index {index}"
    if stack:
        _, idx = stack[0]
        return f"Unclosed opening bracket at index {idx}"
    return "OK"
```

Why store the index? Knowing "it doesn't match" isn't enough — you need to say *where* it went wrong. If the stack only stores characters, popping one tells you nothing about which position it originally came from. So you pack `(char, index)` together — that's the core idea that makes the bracket locator a level up from is_balanced.

Java / AP CSA analogy — use `Deque<Character> stack = new ArrayDeque<>();`, where `push/pop/peek` map to append/pop/[-1].

Bug Comparison Table

WRONG	WHY IT'S WRONG	CORRECT
Closing bracket goes straight to `stack.pop()`	Empty stack pop raises IndexError	Check `if not stack: return False` first
Scan finishes, straight to `return True`	Misses unclosed openers	`return not stack`
Non-bracket characters aren't handled	Letters get pushed as if they were brackets	Only act on actual bracket characters
Locator only stores char	Can't report the error position	Store `(char, index)`

Edge Cases

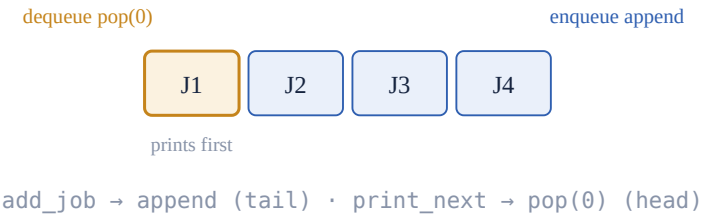
`Empty string → "OK"` `Only openers "((("` `Only closers ")))"` `Nested mix "([{}])"`

◆ EXERCISES

1. `Easy` Implement `is_balanced`, handling only `()[]{}`, skipping letters/digits.
2. `Easy` Given an expression, return the maximum nesting depth of its brackets (hint: track the running max of stack length).
3. `Medium` Complete `check_brackets`, returning one of four possible results, with index.
4. `Medium` BrowserHistory: implement visit / back / forward with **two stacks**; remember that visiting a new page must clear the forward stack.

03 Queue

FIFO — first in, first out. Like waiting in line: first come, first served. The exact opposite of a stack.



`add_job → append (tail) · print_next → pop(0) (head)`

PrinterQueue — the basic queue skeleton — queue

```
class PrinterQueue:
    def __init__(self):
        self.jobs = []

    def add_job(self, job):
        self.jobs.append(job)            # add to the tail

    def print_next(self):
        if not self.jobs:                # edge case: empty queue
            return "No jobs to print"
        return self.jobs.pop(0)          # take from the head (earliest added)

    def current_queue(self):
        return self.jobs
```

Stack vs Queue in one line: both use a list — the only difference is "which end you take from." Stack's `pop()` takes the **tail** (last in, first out); queue's `pop(0)` takes the **head** (first in, first out).
⚠️ Deeper detail: `pop(0)` is O(n) (everything after it has to shift forward). When problems get bigger or performance matters, switch to `popleft()` from `collections.deque`, which is O(1). A plain list is plenty for now.

Java / AP CSA analogy — `Queue<T> q = new LinkedList<>();` → `q.add()` / `q.poll()` / `q.peek()`.

◆ EXERCISES

1. `Easy` Implement PrinterQueue; print_next returns a message when the queue is empty.
2. `Easy` Given a list of job names, simulate printing all of them and return the order printed as a list.
3. `Medium` Add priority: a VIP job is inserted at the head instead of the tail (hint: `insert(0, job)`), everything else stays the same.

04 Linked List (Singly Linked)

Not stored contiguously — each node's `next` points to the next one. **Key mental model: current is a node object; current.value is the actual value.**

The last node's next is always None — that's the signal to stop traversing

◆ Traversal Template (the foundation of everything else)

Node / traversal / display — traversal

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None          # a new node points to nobody by default

# universal traversal skeleton: walk until you hit None
current = self.head
while current is not None:         # ← NOT current.next, or you'll skip the last node
    print(current.value)           # ← .value is the actual value, not current itself
    current = current.next         # ← forget this line = infinite loop
```

◆ Two Kinds of Insert

append (tail insert) vs prepend (head insert) — insert

```
def append(self, value):              # tail insert: walk to the end first
    new_node = Node(value)
    if self.head is None:             # special-case an empty list
        self.head = new_node
        return
    current = self.head
    while current.next is not None:   # find the tail using .next! (not current)
        current = current.next
    current.next = new_node

def prepend(self, value):             # head insert: the order can't be reversed!
    new_node = Node(value)
    new_node.next = self.head         # ① first let the new node grab the old head
    self.head = new_node              # ② then move head over
```

Two "while"s in traversal — don't mix them up: `while current is not None` is for "visiting every node" (display / contains / length); `while current.next is not None` is for "stopping right at the last node" (append finding the tail). The test: do you need to visit that final node itself, or do you need its position?

Delete (three cases)

delete — empty / delete head / delete middle

```
def delete(self, value):
    if self.head is None:                       # ① empty list
        return False
    if self.head.value == value:                # ② deleting the head itself
        self.head = self.head.next
        return True
    current = self.head                         # ③ delete middle/tail: find the "one before it"
    while current.next is not None:
        if current.next.value == value:
            current.next = current.next.next    # skip over the target node
            return True
        current = current.next
    return False
```

Why check current.next.value instead of current.value? A singly linked list can only look forward. To delete a node, you need to be standing *right before* it, so you can wire the previous node's next directly to the one after, skipping the target. That's why the loop checks `current.next.value`, not `current.value`.

Reverse (three pointers)

reverse — prev / current / next_node

```
def reverse(self):
    prev = None
    current = self.head
    while current is not None:
        next_node = current.next   # ① save the way forward first (or you'll lose the rest of the
list)
        current.next = prev        # ② flip the arrow around
        prev = current             # ③ prev moves forward
        current = current.next     # ✗ WRONG! current.next was just overwritten to prev
        current = next_node        # ✓ RIGHT! advance using the next_node you saved earlier
    self.head = prev               # prev ends up pointing at the new head
```

The easiest trap in reverse is right at step ④: you already overwrote `current.next` to point at prev back in step ②, so if you still write `current = current.next`, you'll just spin in place. You must advance using the `next_node` you backed up in step ① — that's the entire reason next_node exists.

Bug Comparison Table

WRONG	WHY IT'S WRONG	CORRECT
Forgot `current = current.next`	Pointer never moves → infinite loop	Must advance current at the end of the loop
`append(current)`	Stores the node object (an address)	`append(current.value)`
display uses `while current.next`	Skips the last node	`while current is not None`
contains returns `False` on the first mismatch	Gives up before checking the rest	Only return False after walking the entire list
prepend moves head before wiring it up	Old head is lost, everything after it disconnects	`new.next=head` first, then `head=new`

Edge Cases

`Empty list, head is None` `Single-node delete / reverse` `Delete the head` `Delete the tail` `Value doesn't exist`

◆ EXERCISES

1. `Easy` Implement `length()` and `display()` (return a plain list).
2. `Easy` `contains(value)`: traverse to find a value — True if found, False after walking the whole list.
3. `Medium` `delete(value)`: cover all four cases — empty / delete head / delete middle / delete tail.
4. `Medium` `reverse()`: implement with three pointers, verify with display afterward.
5. `Medium` Find the **middle node** of a linked list (slow/fast pointers: slow moves one step, fast moves two).

05 Recursion

Every recursive function only asks two questions: **when does it stop (base case)**, and **how does the problem get smaller (recursive case)**.

The two smallest recursions — this is the whole template shape — recursion

```
def factorial(n):
    if n <= 1:                # base case: stop, don't call yourself again
        return 1
    return n * factorial(n - 1)  # recursive case: n shrinks to n-1

def sum_to_n(n):
    if n == 0:                # base case
        return 0
    return n + sum_to_n(n - 1) # problem size shrinks by 1 each time, will eventually bottom out
```

How do you trust that recursion is correct? Don't try to unroll every layer in your head. Just confirm two things: ① the base case will actually be reached (n shrinks each time and won't skip past 0); ② assuming `factorial(n-1)` is already correct, then `n * factorial(n-1)` is correct too. This is the "recursive leap of faith" — trust that the smaller subproblem works, and you're only responsible for assembling this one layer correctly.

Java / AP CSA analogy — nearly identical syntax: `int factorial(int n){ if(n<=1) return 1; return n*factorial(n-1); }`. AP CSA always tests recursion tracing — practice by hand-drawing the call stack.

Bug Comparison Table

WRONG	WHY IT'S WRONG	CORRECT
No base case	Infinite recursion → RecursionError	Write the stopping condition as the first line
`factorial(n)` calls `factorial(n)`	Problem never shrinks, stack overflows	Must call the smaller `factorial(n-1)`
Forgot to `return` the recursive result	Returns None	`return n * factorial(n-1)`

◆ EXERCISES

1. `Easy` Write `count_down(n)`, recursively printing n, n-1, ..., 1.
2. `Easy` Recursively sum all elements in a list (base case: empty list returns 0).
3. `Medium` Recursively reverse a string: `reverse_str("abc") == "cba"`.
4. `Medium` Fibonacci `fib(n)` (two base cases: fib(0)=0, fib(1)=1).

06 Binary Search

Precondition: **the list must already be sorted**. Cuts the search space in half each time — O(log n).

binary_search — memorize these boundaries — search

```
def binary_search(numbers, target):
    left = 0
    right = len(numbers) - 1            # ← it's -1! not len(numbers)
    while left <= right:                # ← <=, not <
        mid = (left + right) // 2
        if numbers[mid] == target:
            return mid
        elif numbers[mid] < target:
            left = mid + 1              # ← +1, skip past mid or you'll loop forever
        else:
            right = mid - 1             # ← -1, skip past mid
    return -1                           # not found
```

Three easy-to-miss points, chained together: `right = len-1` (index of the last element) → `while left <= right` (still need to check when there's a single element left) → `mid±1` (fail to skip past mid and you'll get stuck on the same mid forever). These three come as a package — get one wrong and the whole thing breaks. If you really do get an infinite loop: `Ctrl + C` to force-quit.

Java / AP CSA analogy — same structure; in Java, `mid = (left+right)/2` needs to watch out for integer overflow, so in production you write `left + (right-left)/2`. Python integers have no upper bound, so this isn't a concern.

Bug Comparison Table

WRONG	WHY IT'S WRONG	CORRECT
`right = len(numbers)`	Out of bounds / misses the last element	`right = len(numbers) - 1`
`left = mid` / `right = mid`	Search space never shrinks → infinite loop	`mid + 1` / `mid - 1`
`while left < right`	Skips the case where left==right	`while left <= right`
Forgot `return -1`	Returns None when not found	`return -1` outside the loop

Edge Cases

`Empty list` `Target at the far left / right` `Target doesn't exist` `Single-element list` `Unsorted (precondition violated!)`

◆ EXERCISES

1. `Easy` Implement `binary_search`: find target's index in a sorted list, return -1 if not found.
2. `Easy` Adapt it to return a boolean: whether target exists.
3. `Medium` Find the first occurrence of target's index (when duplicates exist).
4. `Medium` Find the "insert position": return the index where target should be inserted to keep the list sorted (same idea as LeetCode 35).

07 Sorting

Selection → Insertion → Merge, in increasing difficulty. The first two are intuitive $O(n^2)$ approaches; merge sort is $O(n \log n)$ divide-and-conquer.

Selection Sort

Each round, pick the minimum, swap it into position i — $O(n^2)$ · swap

```python
def selection_sort(numbers):
    for i in range(len(numbers)):
        min_index = i                        # what you track is a "position," not a "value"
        for j in range(i + 1, len(numbers)):
            if numbers[j] < numbers[min_index]:
                min_index = j
        numbers[i], numbers[min_index] = numbers[min_index], numbers[i]  # swap into position i
    return numbers
```

The core idea: you swap with `numbers[i]`, not always with `numbers[0]`. And you must track `min_index` (the position) to do the swap — just remembering the minimum value isn't useful on its own.

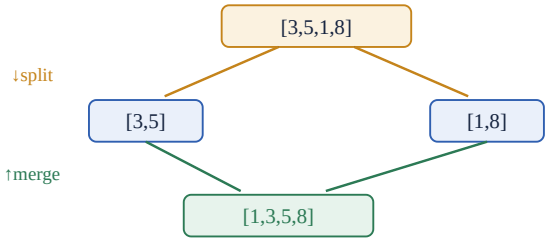
Insertion Sort

Shift version (the textbook way) — O(n²) · shift

```python
def insertion_sort(numbers):
    for i in range(1, len(numbers)):
        key = numbers[i]                    # temporarily hold the value being inserted
        j = i - 1
        while j >= 0 and numbers[j] > key:  # shift anything bigger than key to the right
            numbers[j + 1] = numbers[j]
            j -= 1
        numbers[j + 1] = key                # drop key into the empty slot
    return numbers
```

Shift vs swap: the shift version moves larger elements over as a block and drops key in just once — closer to the textbook and more efficient; the swap version repeatedly swaps the current element with its left neighbor. Both produce the same result — selection sort typically uses swap, insertion sort typically uses shift.

Merge Sort · Focus Review



Split until you can't anymore (length ≤ 1 is naturally sorted) → merge pairs back up from the bottom

merge_sort + merge — O(n log n) · divide & conquer

```python
def merge_sort(numbers):
    if len(numbers) <= 1:           # base case: 0 or 1 elements are naturally sorted
        return numbers
    mid = len(numbers) // 2
    left  = merge_sort(numbers[:mid])    # recursively sort the left half
    right = merge_sort(numbers[mid:])    # recursively sort the right half
    return merge(left, right)            # merge the two already-sorted halves

def merge(left, right):
    result, i, j = [], 0, 0
    while i < len(left) and j < len(right):   # two pointers, pick the smaller each time
        if left[i] < right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[j]); j += 1
    result.extend(left[i:])     # ← append whatever's left over in left (the other side is exhausted)
    result.extend(right[j:])    # ← append whatever's left over in right
    return result
```

Unpacking the points where you got stuck, one by one:
• **Why is the base case len⩽1?** A list with one element is already sorted — nothing to sort. This is the recursion's "ground floor" — once you hit it, you start merging back upward.
• **Why split recursively?** Because merge only knows how to combine *two already-sorted* lists. How do you guarantee the sub-lists are sorted? By running merge_sort on them again. The leap of faith: trust that `merge_sort(left)` returns something sorted — your only job is to merge.
• **Why does merge require left/right to already be sorted?** Because it merges by "two pointers each pointing at the front, always take the smaller one." Only when both sides are sorted is the front guaranteed to be each side's minimum — that's what makes the trick work.
• **What are those two trailing extends for?** The main loop stops as soon as *either side* runs out first. The other side might still have a leftover tail (and it's already sorted), so you just tack the whole remaining chunk onto the end of result.

◆ Bug Comparison Table

WRONG	WHY IT'S WRONG	CORRECT
selection tracks the min value, not its index	Can't swap positions	Track `min_index`
selection always swaps with `numbers[0]`	Only ever sorts the first element	Swap with `numbers[i]`
insertion overwrites before saving key	Shifting wipes out the value being inserted	Save `key = numbers[i]` first
merge is missing the trailing extends	The longer side's tail gets lost	Add both `extend` calls
merge_sort has no base case	Infinite recursion	`if len <= 1: return`

◆ EXERCISES

1. `Easy` Implement selection_sort, and print the list after each round's swap to watch it happen.
2. `Easy` Implement insertion_sort (shift version), then rewrite it as the swap version to compare.
3. `Medium` Implement `merge(left, right)` standalone; verify `merge([3,5],[1,8])` gives `[1,3,5,8]`.
4. `Medium` Complete merge_sort; print left/right at every split so you can watch divide-and-conquer with your own eyes.
5. `Medium` Change merge's comparison operator so the result sorts descending instead.

08 Global Bug Quick-Reference

The highest-frequency errors across all topics, gathered here. When you're stuck, scan this table first.

SYMPTOM	LIKELY CAUSE	WHAT TO CHECK
Program hangs / never finishes	A pointer or range isn't advancing	Linked list: add `current=current.next`; binary search: use `mid±1`
Prints a string like `<__main__...>`	Stored the object, not the value	Switch to `.value`
IndexError	Reading from an empty container	Check for empty first: `if not stack/queue/head`
KeyError	dict does += on a key that doesn't exist yet	`.get(k,0)` or `setdefault`
Result is missing the last element	The while condition used `.next`	Traverse with `while current is not None`
Search always returns False/-1	Returned early before finishing the scan	Put the "return False" *outside* the loop
Sort result is wrong	Confused key/index/min_index	Be clear about what you're comparing (values) vs. what you're rearranging (indices)
RecursionError	Missing base case, or the problem isn't shrinking	Confirm the stopping condition + that the problem size decreases each call

> **Three categories of edge case to always check first:** empty (empty list / dict / stack / queue / linked list), single element (only one item), and ties/duplicates. These three cases cover about 80% of the edge-case bugs across your notes. After writing any function, silently ask yourself: what happens if it's empty? What happens if there's only one?
